THIẾT KẾ KIẾN TRÚC TỔNG THỂ VÀ LỰA CHỌN CÔNG NGHỆ

Đề tài: Hệ thống web quản lý phòng học và thiết bị, hỗ trợ đăng ký và duyệt nhiều cấp

Nhóm thực hiện: Trần Đăng Khoa, Lưu Văn Hưng, Nguyễn Gia Huy

Phiên bản 1.0, ngày 16/10/2026

Tài liệu này trình bày kiến trúc tổng thể và các công nghệ nhóm chọn để xây dựng hệ thống, là sản phẩm của Buổi 4 trong kế hoạch thực hiện đề tài. Nội dung được xây dựng dựa trên tài liệu đặc tả yêu cầu phần mềm phiên bản 1.1 và Product Backlog phiên bản 1.1 của nhóm. Các mã yêu cầu FR, NFR, BR và PB được dùng thống nhất với hai tài liệu này.

CHƯƠNG 1: KIẾN TRÚC TỔNG THỂ

1.1. Mô hình kiến trúc

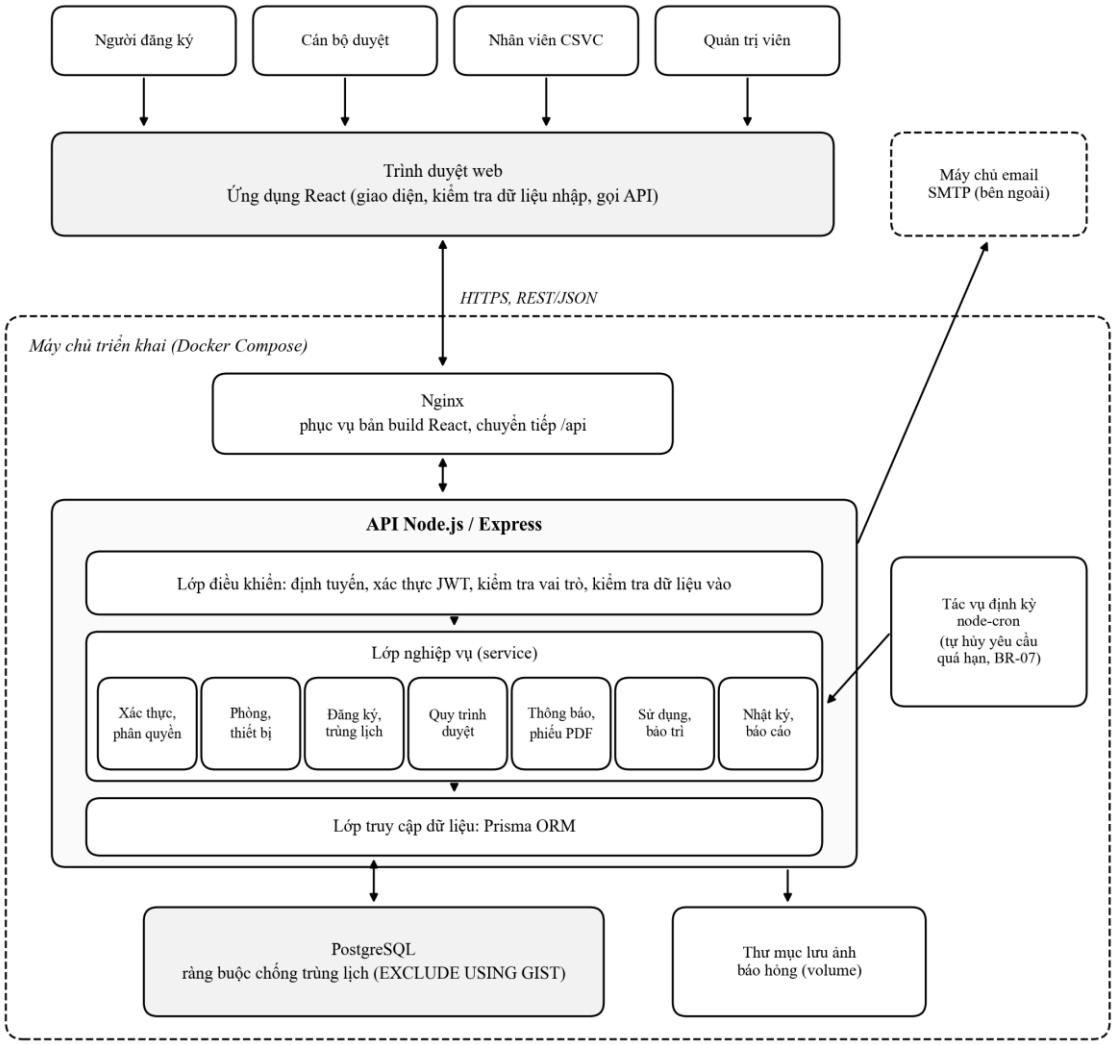
Hệ thống được xây dựng theo mô hình khách và máy chủ. Phía người dùng là một ứng dụng web một trang viết bằng React, chạy trên trình duyệt và giao tiếp với máy chủ qua giao diện lập trình REST, dữ liệu trao đổi ở dạng JSON. Phía máy chủ là một ứng dụng Node.js duy nhất, bên trong chia thành các module theo nghiệp vụ. Dữ liệu được lưu trong cơ sở dữ liệu PostgreSQL.

Bên trong máy chủ, mã nguồn được tổ chức theo kiến trúc phân lớp gồm ba lớp. Lớp điều khiển nhận yêu cầu, xác thực người dùng, kiểm tra quyền và kiểm tra dữ liệu đầu vào. Lớp nghiệp vụ chứa toàn bộ quy tắc xử lý như kiểm tra trùng lịch hay xác định bước xác nhận. Lớp truy cập dữ liệu là nơi duy nhất làm việc với cơ sở dữ liệu. Cách chia này giúp từng thành viên làm việc trên module của mình mà ít ảnh hưởng đến phần của người khác, đồng thời dễ viết kiểm thử cho lớp nghiệp vụ.

Nhóm không chọn kiến trúc microservices. Với quy mô khoảng 22 yêu cầu chức năng, ba thành viên và hai sprint xây dựng, việc tách thành nhiều dịch vụ riêng sẽ làm tăng đáng kể công sức triển khai, giám sát và xử lý giao dịch giữa các dịch vụ mà không mang lại lợi ích tương xứng. Một ứng dụng chia module rõ ràng vẫn có thể tách dần nếu sau này hệ thống được mở rộng.

1.2. Sơ đồ kiến trúc

Các thành phần chính của hệ thống và luồng giao tiếp giữa chúng được thể hiện trong Hình 1.1.



Hình 1.1. Sơ đồ kiến trúc tổng thể của hệ thống

Người dùng ở cả bốn vai trò đều truy cập qua trình duyệt. Trên máy chủ triển khai, Nginx phục vụ các tệp giao diện đã được build sẵn và chuyển tiếp các yêu cầu có đường dẫn bắt đầu bằng /api tới ứng dụng Node.js. Ứng dụng Node.js đọc và ghi dữ liệu vào PostgreSQL, lưu ảnh báo hỏng vào một thư mục được gắn ra ngoài container và gửi thư thông báo qua máy chủ email bên ngoài. Một tác vụ định kỳ chạy cùng ứng dụng để tự hủy các yêu cầu đã đến giờ sử dụng mà chưa được duyệt xong theo quy tắc BR-07.

1.3. Các module nghiệp vụ

Lớp nghiệp vụ được chia thành bảy module. Mỗi module gắn với một nhóm yêu cầu chức năng và các mục backlog tương ứng như trong Bảng 1.1. Cách chia này cũng là căn cứ để nhóm phân công công việc trong từng sprint.

Bảng 1.1. Các module nghiệp vụ

Module	Chức năng chính	Yêu cầu	Sprint
Xác thực, phân quyền	Đăng nhập, cấp và kiểm tra JWT, phân quyền theo vai trò, quản lý tài khoản	FR-01, FR-02	Sprint 2
Phòng, thiết bị	Quản lý phòng, thiết bị theo phòng, tình trạng thiết bị, sinh mã QR	FR-03 đến FR-05	Sprint 2, 5
Đăng ký, trùng lịch	Tra cứu phòng trống, lập và gửi yêu cầu, lưu nháp, sửa và hủy, kiểm tra trùng lịch, lịch sử dụng phòng	FR-06 đến FR-11	Sprint 2, 3
Quy trình duyệt	Xác định bước xác nhận theo BR-03 đến BR-06, duyệt hoặc từ chối có lý do, cấu hình quy tắc	FR-12, FR-13, FR-16	Sprint 3
Thông báo, phiếu PDF	Thông báo trong hệ thống, gửi email, xuất phiếu đăng ký đã duyệt	FR-14, FR-15	Sprint 4, 5
Sử dụng, bảo trì	Bàn giao, nhận trả, check-in bằng mã QR, báo hỏng, phiếu bảo trì	FR-17 đến FR-20	Sprint 5
Nhật ký, báo cáo	Ghi nhật ký thao tác, thống kê tần suất sử dụng phòng và thiết bị hỏng	FR-21, FR-22	Sprint 4

Các chức năng FR-11, FR-16, FR-20 và FR-22 thuộc nhóm mở rộng trong backlog, chỉ được làm khi các yêu cầu mức Must và Should đã hoàn thành. Kiến trúc vẫn chừa sẵn vị trí cho các chức năng này để không phải thay đổi cấu trúc khi bổ sung.

1.4. Luồng xử lý một yêu cầu đăng ký

Khi người đăng ký gửi yêu cầu, giao diện React kiểm tra sơ bộ các trường bắt buộc rồi gọi API tạo yêu cầu. Lớp điều khiển xác thực mã JWT, kiểm tra vai trò người gửi và kiểm tra lại dữ liệu đầu vào. Lớp nghiệp vụ kiểm tra thời hạn đặt trước theo BR-02, sau đó kiểm tra trùng lịch với các lượt đặt đã được duyệt của cùng phòng theo BR-01.

Nếu không trùng, module quy trình duyệt áp dụng các quy tắc BR-03 đến BR-06 để xác định các bước xác nhận và đơn vị xử lý đầu tiên. Yêu cầu, các bước xác nhận và dòng nhật ký được ghi trong cùng một giao dịch cơ sở dữ liệu, vì vậy nếu một bước lỗi thì toàn bộ thao tác được hủy. Khi bước cuối cùng được duyệt, trạng thái lượt đặt chuyển sang đã duyệt. Tại thời điểm này ràng buộc trong PostgreSQL kiểm tra lần cuối để bảo đảm không có hai lượt đặt đã duyệt chồng giờ trên cùng một phòng.

1.5. Quyết định kiến trúc theo yêu cầu chất lượng

Mỗi yêu cầu chất lượng quan trọng trong tài liệu đặc tả được nhóm gắn với một quyết định kiến trúc cụ thể, kèm theo đánh đổi mà nhóm chấp nhận, như trong Bảng 1.2.

Bảng 1.2. Quyết định kiến trúc theo yêu cầu chất lượng

Yêu cầu	Quyết định	Đánh đổi
NFR-01: không có lượt đặt đã duyệt chồng giờ	Dùng ràng buộc loại trừ EXCLUDE USING GIST trên cặp mã phòng và khoảng thời gian, chỉ áp dụng cho lượt đặt đã duyệt [1]	Phụ thuộc vào tính năng riêng của PostgreSQL, cần extension btree_gist và một phần migration viết bằng SQL
NFR-02: kiểm tra trùng lịch không quá 1 giây	Ràng buộc trên tạo sẵn chỉ mục GiST, phép kiểm tra trùng chỉ là một truy vấn có chỉ mục	Thao tác ghi chậm hơn một chút do phải cập nhật chỉ mục
NFR-03: trang tra cứu tải không quá 3 giây	Lọc và phân trang ở máy chủ, Nginx nén và lưu bộ nhớ đệm tệp giao diện	Lần tải đầu tiên của ứng dụng một trang nặng hơn trang web thường
NFR-04: kiểm tra quyền, băm mật khẩu	Mọi route nhạy cảm đi qua middleware kiểm tra JWT và vai trò, mật khẩu băm bằng bcrypt	Phải xử lý hết hạn và làm mới mã đăng nhập
NFR-06: chạy trên máy tính và điện thoại	Dùng bộ giao diện Ant Design có lưới bố cục co giãn	Giao diện mang phong cách chung của thư viện
NFR-07: ghi nhật ký mọi thao tác duyệt, sửa, hủy	Ghi nhật ký ở lớp nghiệp vụ, trong cùng giao dịch với thao tác chính	Mỗi thao tác có thêm một lần ghi vào cơ sở dữ liệu

1.6. Môi trường triển khai

Toàn bộ hệ thống được đóng gói bằng Docker và khởi chạy bằng Docker Compose với ba container. Container thứ nhất chạy Nginx cùng bản build của ứng dụng React. Container thứ hai chạy ứng dụng Node.js cùng tác vụ định kỳ. Container thứ ba chạy PostgreSQL. Dữ liệu cơ sở dữ liệu và thư mục ảnh được lưu trên volume nên không mất khi khởi động lại container.

Các thành viên dùng cùng tệp cấu hình Docker Compose để chạy hệ thống trên máy cá nhân, nhờ đó môi trường phát triển giống với môi trường demo. Trước buổi demo, nhóm nạp sẵn dữ liệu mẫu gồm phòng, thiết bị và mỗi vai trò một tài khoản.

CHƯƠNG 2: LỰA CHỌN CÔNG NGHỆ

2.1. Tiêu chí lựa chọn

Nhóm chọn công nghệ dựa trên bốn tiêu chí. Thứ nhất, công nghệ phải đáp ứng được các yêu cầu trong tài liệu đặc tả, nhất là chống trùng lịch và phân quyền. Thứ hai, frontend và backend dùng chung ngôn ngữ JavaScript để ba thành viên dễ hỗ trợ nhau khi một phần bị chậm tiến độ. Thứ ba, công nghệ phải miễn phí, có tài liệu chính thức đầy đủ và cộng đồng sử dụng rộng. Thứ tư, hệ thống phải chạy được trên máy cá nhân và đóng gói được bằng Docker để demo ổn định.

2.2. Bảng công nghệ

Các công nghệ được chọn cho từng thành phần, lý do chọn và phương án thay thế nhóm đã cân nhắc được trình bày trong Bảng 2.1.

Bảng 2.1. Công nghệ sử dụng

Thành phần	Công nghệ	Lý do chọn	Phương án thay thế
Giao diện	React, Vite	Chia giao diện thành component dùng lại được, Vite build nhanh khi phát triển	Vue.js
Bộ giao diện	Ant Design	Có sẵn bảng, biểu mẫu, lịch và lưới co giãn, phù hợp màn hình quản lý	Material UI
Định tuyến, gọi API	React Router, Axios	Quản lý các trang theo vai trò, gắn mã JWT vào mọi yêu cầu	Fetch API
Máy chủ ứng dụng	Node.js bản LTS, Express	Dùng chung JavaScript với giao diện, Express gọn và dễ tổ chức theo lớp	NestJS
Truy cập dữ liệu	Prisma ORM	Định nghĩa bảng và quan hệ trong một tệp schema, có công cụ migration	Sequelize
Cơ sở dữ liệu	PostgreSQL	Hỗ trợ kiểu khoảng thời gian và ràng buộc loại trừ để chống trùng lịch, có giao dịch đầy đủ [1]	MySQL
Xác thực	JWT, bcrypt	Không cần lưu phiên trên máy chủ, mật khẩu được băm trước khi lưu	Session và cookie

Thành phần	Công nghệ	Lý do chọn	Phương án thay thế
Tác vụ định kỳ	node-cron	Chạy việc tự hủy yêu cầu quá hạn ngay trong ứng dụng, không cần dịch vụ riêng	Lịch của hệ điều hành
Gửi email	Nodemailer	Gửi thư qua máy chủ SMTP có sẵn, cấu hình đơn giản	Dịch vụ gửi thư trả phí
Phiếu PDF, mã QR	PDFKit, qrcode	Tạo phiếu đăng ký và mã QR trực tiếp trên máy chủ	Puppeteer
Tải ảnh lên	Multer	Nhận ảnh báo hỏng, giới hạn dung lượng và định dạng tệp	Lưu trữ đám mây
Kiểm thử	Jest, Supertest	Kiểm thử đơn vị lớp nghiệp vụ và kiểm thử tích hợp API	Mocha
Triển khai	Docker, Docker Compose, Nginx	Chạy giống nhau trên mọi máy, một lệnh khởi động toàn bộ hệ thống	Cài đặt trực tiếp

2.3. Công cụ hỗ trợ

Ngoài các công nghệ dùng trong sản phẩm, nhóm dùng thêm một số công cụ để quản lý mã nguồn, tài liệu và tiến độ như trong Bảng 2.2.

Bảng 2.2. Công cụ hỗ trợ

Công cụ	Mục đích sử dụng
Git, GitHub	Quản lý mã nguồn chung, mỗi thành viên làm trên nhánh riêng và gộp vào nhánh chính qua pull request
Visual Studio Code	Soạn thảo mã nguồn cho cả frontend và backend
Postman	Thử và lưu bộ yêu cầu mẫu cho các API trong quá trình phát triển
draw.io	Vẽ sơ đồ kiến trúc, sơ đồ quy trình và sơ đồ cơ sở dữ liệu
Microsoft Excel	Theo dõi Product Backlog, kế hoạch sprint và checklist công việc
Docker Desktop	Chạy hệ thống bằng Docker Compose trên máy cá nhân

TÀI LIỆU THAM KHẢO

[1] The PostgreSQL Global Development Group, "PostgreSQL 18 Documentation, 8.17. Range Types", https://www.postgresql.org/docs/current/rangetypes.html, truy cập ngày 16/10/2026.